Assignment 1 (Point Operations: Intensity and Geometry)

Instructions for Assignment 1:

- Download Matlab.zip or Python.zip according to your preference from the link.
- The folder structure (**strictly don't change**) once you unzip is as follows:
  - Files 'Que1' to 'Que5' are the main codes for running your codes. **Don't alter the contents of these files**. If you are finding any errors, please raise the issue through discussion forum.
  - The folder 'Data' has some sample data for each of the questions. Do note that, you may have to collect your own data which has weightage for the assignment.
  - The folder 'Functions' has been made for you and this is the main folder where you will find functions corresponding to different questions. **Please edit the respective functions**.
  - The folder 'Results' will automatically populate results once you run the files 'Que1' to 'Que5'.
- The distribution of marks for the assignment 1 is as follows: 30% for data acquisition (your own dataset), 50% for functionality and 20% for clean code and comments.

Questions:

1. Write a function which takes an input image (gray scale) and automatically do contrast stretching, histogram equalization and gamma correction (if needed) of the image. The output of the function should be the processed image. ***Strictly make use of autoHist.m in Matlab/Functions or autoHist.py in Python/Functions to make your function. Also use Que1.m or Que1.py for evaluating your results. Currently the image being read is 'Data\flower.jpg' and please feel free to add your own images which will carry additional weightage.***

2. Using an appropriate image processing (only point intensity operations covered in Lectures), write a universal function which could do the following (a and b). Please note the input to the function will be a video which has a static background and a moving foreground, and the output will be a video and an image as given below:
   a. A single output image which captures mostly the background without the moving part.
   b. A single video output which shows the moving part in each of the frames with reduced background effect.
   c. Illustrate the working of the developed code by collecting your own data. For example, using your smartphone camera record a scene which has a static part and a moving part (e.g. rotation of a fan or a moving ball against static back ground).

   Strictly make use of motionSegmentation.m in Matlab/Functions or motionSegmentation.py in Python/Functions to make your function. Also use Que2.m or Que2.py for evaluating your results. We have given a sample file motionSegmentation.mp4 for your usage. However, it is important that you show the results using your own data and rename your data to motionSegmentation.mp4

3. Read the image 'grain.png' available in the directory. It can be seen that the image is dark towards left and white towards right. Using image addition, subtraction and histogram-based point operations, try to obtain a better image. The input to the function will be an image and the output will be the processed image. ***Strictly make use of grain.m in Matlab/Functions or grain.py in Python/Functions to make your function. Also use Que3.m or Que3.py for evaluating your results.***

4. Write a MATLAB/Python function for computing an affine transform of an image. The input to your function is: (i) image, (ii) 3D spatial transformation matrix T of the form $\begin{bmatrix} a & b & 0 & t_x \\ c & d & 0 & t_y \\ 0 & 0 & 1 & 0 \end{bmatrix}$, and (iii) interpolation method (nearest neighbourhood, linear and bi-linear) and output will be the transformed image. Test you function as follows:
   a. Spatial transformation of the input image using the matrix T and the bilinear interpolation, where T is specified as T = [0.3 0.1 0; 0.5 1.9 0; 0 0 1].
   b. Observe the difference between the input image and the image that has been obtained by applying the inverse spatial transform T?

   Strictly make use of affineTransform.m and invaffineTransform.m in Matlab/Functions or affineTransform.py and invaffineTransform.py in Python/Functions to make your functions. Also use Que4.m or Que4.py for evaluating your results. We have given a sample file grain.png for your usage. However, feel free to use any image for reference.

5. Using your smartphone camera record a static scene while shaking the camera mildly. Using affine transformation algorithm code developed in (4), remove the shaking effect of the camera. Also estimate the approximate motion induced by the shaking. (Hints: Identify a set of points (name it as key points) on the first frame of the image. Try to align the "key points" identified in the first frame for all the frames in the image using affine transformation). The input to the function will be the recorded video and the outputs will be a video without camera motion and a signal showing the estimation (approx.) of motion.

   Strictly make use of motionEstimation.m in Matlab/Functions or motionEstimation.py in Python/Functions to make your function. Also use Que5.m or Que5.py for evaluating your results. We have given a sample file motionEstimation.mp4 for your usage. However, it is important that you show the results using your own data and rename your data to motionEstimation.mp4

Please follow the below guidelines while submitting the assignment:

- Zip back the folder after you are done with the assignment and name the zip file with your Roll Number. Upload the same to Moodle and if there are size restrictions, please upload in your Google drive and share the link in a text file and upload in Moodle. **Uploading in Moodle is mandatory.**
- Codes will be evaluated using an automation tool and counter checked for correctness. Hence it is important to follow the above guidelines correctly.